Client Report: Income Classification and Customer Segmentation

Likith Palabindela
likith.palabindela@jpmchase.com

1 Introduction

To complete this project, I've taken several steps to demonstrate my intuition and approach as a data scientist. As a part of the project, the client wants to learn what income class a person belongs to, given 40 employment and demographic related variables. There are 2 parts to this project – a classification model and a segmentation model. The classification model will be useful to classify someone as having an income below $50,000 or above $50,000. The segmentation model is useful for learning more about the different kinds of people in the dataset and how it relates to income. This report is structured as follows: a common exploratory data analysis section and, for each model, sections on preprocessing, model architecture, the training algorithm, the evaluation procedure, and findings. In each of the sections, I will explain my approach and reasoning, while in the findings section, I will talk specifically about what decisions to make as the client depending on your objectives.

2 Data Exploration

This section is common for both models as the exploratory data analysis (EDA) was done to get a better feel for the data and is model agnostic. We have a dataset of size ∼200,000 rows by 42 columns, of which one column is the label and one is the weight. The first thing to be noted is that the labels in the dataset are heavily imbalanced. About 93.8% of the data makes less than $50,000, while only 6.2% of the data makes $50,000 or more. This is an important point as it brings us to how we measure the performance of our model. If our model predicts 0 for every row in the dataset, our model would be 93.8% accurate, which isn't helpful. To help with this data imbalance, we are provided with a weight column. The weight for a row tells us the relative number of people in the population that the row represents. This is incredibly important as we can use it to downsample or upsample either of our classes.

Next, I identified columns in the dataset that might not be meaningful. There are 4 columns where about 50% of the rows have "?", namely: *migration code-change in msa*, *migration code-move within reg*, *migration code-change in reg*, and *migration prev res in sunbelt*. These features will not be useful in training the model as only half of the rows contain meaningful values. Another value to account for is "Not in universe". This happens when the person does not meet the criteria for the question. For the following columns: *fill inc questionnaire for veteran's admin*, *reason for unemployment*, *enroll in edu inst last wk*, *state of previous residence*, *region of previous residence*, *member of a labor union*, *family members under 18*, *live in this house 1 year ago*, *major industry code*, *major occupation code*, *class of worker*, and *migration prev res in sunbelt*, the percentage of "Not in universe" values ranges from 50%–90%, indicating that these features don't provide much valuable insight into income. Data is valuable and removing features can be very detrimental. To confirm these findings, I trained an XGBoost model to identify the top 15 most important features. Only 3 of the columns mentioned previously appear in this list: *major industry code*, *major occupation code*, and *class of worker*. The remaining columns were not used in the final list of variables.

After identifying irrelevant columns, we need to identify important columns – those that are correlated with income level. There are 2 correlation tests I used: the Mann-Whitney U test and Cramér's V. The Mann-Whitney U test compares differences in distributions, while Cramér's V measures the strength of association for nom-

inal data. The 5 most important variables from the Mann-Whitney test are *age*, *wage per hour*, *capital gains*, *capital losses*, and *dividends from stocks*. The top 10 for Cramér's V are *detailed occupation recode*, *education*, *major occupation code*, *detailed industry recode*, *major industry code*, *class of worker*, *detailed household and family stat*, *tax filer stat*, *detailed household summary in household*, and *marital stat*. These are variables that we will likely use in our initial models.

3 Classification Model

3.1 Preprocessing

The first step I took in the preprocessing pipeline is replacing missing values and "?" values with the string "Unknown". There was only one column with missing values (*hispanic origin*). The next step is identifying numeric and categorical columns. Columns of type `int64` or `float64` were numeric columns, and columns of type `str` were categorical. Four columns in particular (*detailed industry recode*, *detailed occupation recode*, *own business or self employed*, *veterans benefits*) are of type `int64`, but the values are codes – meaning they are categorical in nature. The labels were converted to classes (<$50,000 being class 0, >=$50,000 being class 1). To avoid data leakage, the labels and weights will not be a part of the feature set. The weights, however, will be used during training.

Next, I engineer features that are not included in the provided dataset. There are 5 engineered features used in the model. *Annual_labor_proxy* combines wage per hour and weeks worked to approximate total labor income. *Has_capital_gains* and *has_dividends* convert numeric variables that are skewed by zeros to boolean variables for profit indicators. *Net_capital* takes into account capital losses. *Full_year_worker* is included since the number of hours worked is correlated with income.

Moving onto feature selection, a number of features are not included in the final list due to containing a high number of "Unknown" values, being highly correlated with other features, or because engineered features are a better choice. The final numerical features are: *age*, *weeks worked in year*, *dividends from stocks*, and *num persons worked for employer*. The final categorical features are: *education*, *sex*, *tax filer stat*, *detailed occupation recode*, *detailed industry recode*, and *class of worker*. Standardization was applied to numerical variables and target encoding to categorical variables. Target encoding was chosen as it performed better than ordinal encoding and did not drastically increase the dimensionality of the data like one-hot-encoding. A 70/30 train/test split was applied with stratification on the target label. The feature selection process was iterative, experimenting with combinations of features and model types before settling on a final list.

3.2 Model Architecture

To thoroughly evaluate model performance, six models were trained and compared in total. For the purposes of this report, I will focus on three types of models: logistic regression, random forest, and XGBoost – which best illustrate the progression from simple to complex. Logistic regression provides a transparent baseline and tests whether income is linearly separable. A standard architecture was used with `class_weight` set to `balanced`. The features with the highest coefficients were analyzed and played a part in the final list of variables. Scores and metrics for each model are discussed in the evaluation procedure section.

There are 2 different ensemble models to evaluate: bagging models and boosted models. Bagging models train multiple trees in parallel aiming to reduce model variance, while boosting models train trees sequentially, aiming to reduce bias. For the random forest model, I used 200 estimators, `max_depth` of 20, `min_samples_leaf` set to 5, and `class_weight` set to `balanced`. For the XGB model, parameters were set to 8 for `max_depth`, 0.05 for `learning_rate`, and 200 for `n_estimators`, found via grid search. Although tree-based models are strongly suited for tabular data, an MLP was also trained to test whether deeper representation learning improves performance. The MLP used `hidden_layer_sizes` of (128, 64), `relu` activation, `adam` solver, `alpha` of 0.0001, `batch_size` of 256, and `max_iter` of 50. These values were not fine-tuned as the MLP was included for experimental comparison only.

3.3 Training Algorithm

This report focuses on 3 models: logistic regression, random forest, and XGB. Each model was trained on the same preprocessed training set us-

Model	ROC-AUC	Precision	Recall	F1	Threshold
Logistic Regression	0.942	0.28	0.88	0.42	0.5
Random Forest	0.947	0.42	0.76	0.55	0.5
XGB	0.945	0.65	0.50	0.57	0.5
XGB (w/ SMOTE)	0.937	0.15	0.98	0.26	0.5
XGB (Tuned)	**0.953**	**0.747**	0.482	**0.586**	0.5
MLP	0.916	0.56	0.51	0.54	0.5

Table 1: Classification model performance on the weighted test set. Metrics are reported for the positive class (income $> \$50K$).

ing the stratified split described above. When supported by the sklearn API, sample weights were incorporated during training to reflect the Census data population weighting. Class imbalance was addressed using `class_weight = balanced` for logistic regression and random forest. To offset the highly imbalanced dataset, I also attempted SMOTE (Synthetic Minority Oversampling Technique), which creates new synthetic samples for the minority class to make the dataset more balanced. The XGB model was evaluated against both weighted training and SMOTE. For the MLP, no balance correction was provided.

3.4 Evaluation Procedure

Our dataset is heavily imbalanced, so accuracy would not be a helpful metric. To measure performance, I chose ROC-AUC, which evaluates binary models by plotting the True Positive Rate against the False Positive Rate at different thresholds. In addition, I examined precision $\frac{TP}{TP+FP}$ and recall $\frac{TP}{TP+FN}$ for class label 1. Precision tells us what fraction of those predicted as high earners actually are; recall tells us what fraction of all true high earners our model correctly identifies. All models were evaluated on the same held-out 30% test set. ROC-AUC and classification metrics were computed using the sample weights so performance reflects the real population distribution. Threshold tuning is explored further in the findings section. The trade-offs between precision, recall, and threshold selection, as well as overall ranking performance, are visualized in Figure 2.

3.5 Findings

The metric findings are summarized in Table 1. Across all six models, ROC-AUC ranges narrowly from 0.916 to 0.953, with the tuned XGB model achieving the highest score. These findings are important for two reasons: first, there is a minuscule difference between the baseline AUC and the final model's AUC, and second, it raises the question of whether added model complexity is justified given the client's specific objectives. If the client prefers an interpretable model to explain why a particular person was targeted, the difference in AUC is negligible and I would recommend logistic regression. If maximizing AUC and precision is the priority, the XGB model results in the fewest wasted contacts, making it the most cost-efficient model when marketing is expensive. The added complexity of tree models and feature engineering earns diminishing returns on this dataset – the features themselves are doing most of the work, not the model complexity.

The model can always be tuned according to the client's needs, which takes us to the precision-recall trade-off. If the cost of reaching a person is high, increasing the threshold raises precision – the model would be more successful at predicting high earners and the client would lose less money targeting the wrong people, but recall would decrease. If the cost of targeting people is cheap (email/digital ads), lowering the threshold increases recall – the marketing would cover a larger population of high earners at the cost of some incorrect predictions.

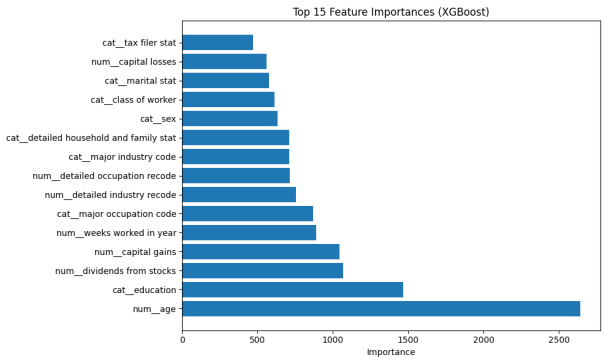

Figure 1: Top engineered and selected features ranked by model importance.

4 Segmentation Model

4.1 Preprocessing

To clean the data, I first removed records with age less than 18 since minors are not a relevant marketing audience. Including minors artificially inflated the clustering scores by forming a trivially separable group. "?" values were replaced with "Unknown" so they are treated as a separate category during encoding. Capital gains, dividends, and wage per hour are heavily skewed by zeros. A log transform was applied to these columns to compress the skewed distribution and make them usable for distance-based clustering. A *has_investments* variable was created as a flag to capture whether the person has any gains, losses, or dividends – this separates the magnitude of the investment from whether they have any investments. Wage per hour is given in cents for hourly workers and is zero for salaried workers, so the log transform brings both populations onto a similar scale.

Features were selected and grouped into four intuitive categories: demographic, employment, education, and financial. Raw capital gains, dividends, and wage per hour were excluded from clustering since their log-transformed counterparts are better representations. The label and weight columns were excluded to prevent data leakage. Numerical features were standardized using `StandardScaler` so that features with large ranges do not dominate distance calculations. Target encoding was applied to categorical features, replacing each category with its mean income rate. This introduces some label influence into the segmentation, which is acceptable given that the client's primary interest is in identifying segments that differ meaningfully in income level. Following encoding and scaling, PCA was applied to reduce the feature space to 5 components explaining 90% of the variance.

4.2 Model Architecture

The primary model I explored was K-Means clustering. K-Means is an iterative unsupervised algorithm that works by minimizing distance to the cluster centroids. It is suitable for this problem because we are working with a large, tabular dataset where interpretability is essential. While K-Means could be run on the encoded features directly, PCA helps with noise reduction, decorrelation, and computational efficiency. The 90% variance threshold is standard practice and offers a good balance between information loss and dimensionality reduction. To select the appropriate k value, I evaluated the model across a range ($k = 2$–9) and found that $k = 5$ produced the best score and interpretable segments.

Some limitations of K-Means are that it assumes the data forms spherical clusters of equal size, which may not perfectly reflect our population. We must specify k upfront, which is why I evaluated across a range of values. Results can also vary with centroid initialization, so I set `n_init = 20`. In addition to K-Means, I also tried hierarchical clustering and DBSCAN. The results are summarized in the findings section.

4.3 Training Algorithm

Unlike the classification section, there is no train/test split. Clustering is unsupervised and uses the full dataset – there is no label being predicted, so there is no risk of overfitting. The silhouette score was computed on a random subset of 20,000 records rather than the full dataset purely for computational efficiency; this is standard practice and does not affect the result. Sample weights were incorporated during clustering to ensure cluster centroids reflect the real population distribution rather than the survey sample. K-Means iterates until the cluster assignments for each record stop changing or a maximum number of iterations is reached. The default maximum of 300 iterations is sufficient for this dataset.

4.4 Evaluation Procedure

The primary metric used to measure performance is the silhouette score. This measures how similar a record is to its own cluster compared to other clusters, ranging from -1 to 1. A score of 1 indicates that points are far from neighboring clusters and close to their own, while a score of 0 indicates overlapping clusters. A score of -1 may indicate that points are assigned to the wrong clusters. Silhouette score was chosen as a standard way of evaluating clustering quality. A key limitation is that a higher silhouette does not necessarily mean better segments for the business – as demonstrated by the inflated scores when minors were included. Segment quality was also evaluated qualitatively by inspecting cluster profiles for meaningful distinctiveness. Since income labels are available, they were used post-hoc to validate segment findings. Hierarchical clustering and DBSCAN were

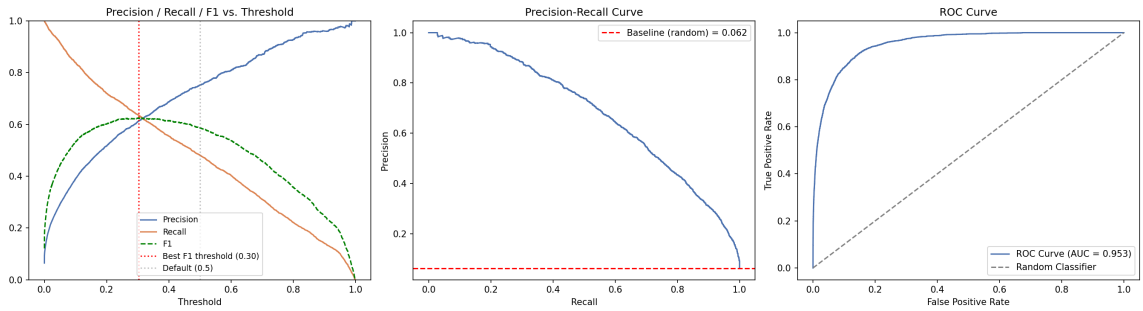

Figure 2: Threshold analysis and model evaluation curves. (Left) Precision, recall, and F1 score as functions of classification threshold, with the optimal F1 threshold and default 0.5 threshold marked. (Center) Precision–Recall curve with baseline equal to the population positive rate (6.2%). (Right) ROC curve showing overall class separability (AUC = 0.953).

used as secondary validation: if independent algorithms find similar structure, confidence in the K-Means result increases.

4.5 Findings

The silhouette peaks at both $k = 5$ and $k = 7$ with a difference of 0.0003. For simplicity and interpretability, we selected 5 clusters. The findings are summarized in Table 2. Income rates range from 1% to 33% across segments against a population average of 6.2%, which is compelling evidence that the clusters are economically meaningful.

Everyday workers and hourly workers are distinguished by pay structure rather than hours – both work approximately 46 weeks full time, but one is salaried and one is hourly paid. The educated earners cluster contains people with bachelor's degrees and high dividend income. The high earners are the highest-value cluster at 33% high income, distinguished from educated earners by the simultaneous presence of capital gains and employment income, suggesting active wealth accumulation rather than passive income drawdown.

High earners and educated earners should be the primary targets for the client, while everyday workers, hourly workers, and the retired are lower priority. The client should target high earners and educated earners with premium products and financial services, where the high cost per contact is justified by high conversion likelihood. The other three populations should be targeted with price-sensitive promotional offers and value messaging at a lower cost per contact.

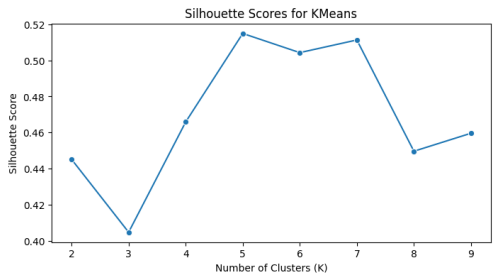

Figure 3: Silhouette scores across different values of k for K-Means clustering.

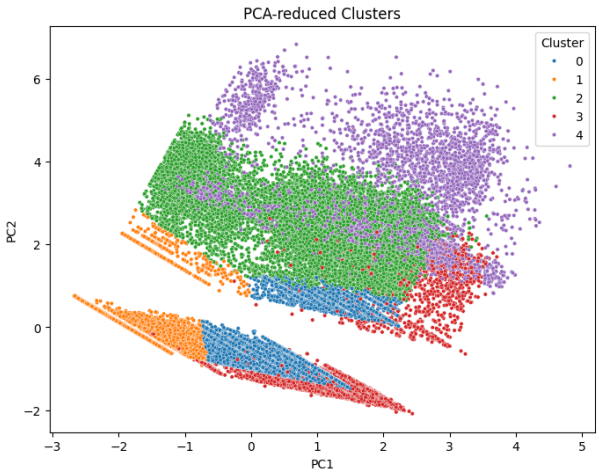

Figure 4: 2D PCA projection of the five selected clusters. Clear separation indicates meaningful segmentation structure.

Hierarchical clustering consistently found similar structure across the population when tuned with the proper k. After tuning with the proper `eps` value, DBSCAN also divided the population into 5 clusters. Consistency across models increases confidence that the K-Means structure reflects genuine patterns in the data. The primary limitation of this model is the age of the data – the client would need to retrain the model with more recent data since income distributions and demographic patterns have shifted significantly in the

Segment	Age	Weeks	Wage/hr	Cap Gains	Dividends	% $> \$50K$	Key Characteristics
Everyday Workers	38	46	$0	$0	$0	8%	Full-time salaried workers, clerical/admin roles, no investment activity
Retired	55	2	$0	$0	$0	1%	Minimal employment, no investments, lowest income segment
Educated Earners	51	34	$0.08	$0	$1,799	23%	Bachelor's degree, dividend income, part-time employment
Hourly Workers	37	46	$9.87	$0	$34	4%	Hourly-paid full-time workers, limited financial assets
High Earners	49	40	$0.78	$11,814	$1,035	**33%**	Strong employment income plus significant capital gains and dividends

Table 2: Summary statistics and defining characteristics of the five selected clusters. Income rate varies significantly across segments relative to the 6.2% population average.

past 30 years. The client should also monitor data drift and identify how scores and clusters change with newer data.

5 Conclusion

In conclusion, this project successfully trained an income classification model achieving a ROC-AUC of 0.953 and a segmentation model identifying five distinct customer groups. The most interesting finding across both models is that the income prediction problem is largely linearly separable — logistic regression achieves a ROC-AUC of 0.942 with minimal feature engineering, and the gap to the best model is minuscule. Overlaying both models gives the client the most value: the classifier identifies who is likely a high earner, while the segmentation model identifies what kind of person they are, enabling differentiated targeting and messaging per segment. There are several improvements I would explore given more time. The most impactful would be k-fold cross-validation, as the reported metrics are based on a single 70/30 split and could be sensitive to how the data was split. Cross-validation would provide a more reliable estimate of generalization performance. I would also explore weighted undersampling as an alternative to class weights to see whether it outperforms the current approach. For feature selection, permutation importance would provide a more principled and reproducible approach than the iterative manual method I used. Finally, for the segmentation model, Gaussian Mixture Models would be a natural next step as they produce soft cluster assignments, giving each person a probability of belonging to each segment rather than a hard assignment. The primary limitation of both models is the age of the data — retraining on more recent Census data would be the single most impactful practical improvement available to the client.

References

Tianqi Chen and Carlos Guestrin. Introduction to boosted trees — XGBoost documentation. https://xgboost.readthedocs.io/en/stable/tutorials/model.html, 2024.

GeeksforGeeks. Handling imbalanced data for classification. https://www.geeksforgeeks.org/machine-learning/handling-imbalanced-data-for-classification/, 2024.

Ron Kohavi. Census income. UCI Machine Learning Repository, 1996. URL https://archive.ics.uci.edu/dataset/20/census+income.

F. Pedregosa et al. Clustering — scikit-learn documentation. https://scikit-learn.org/stable/modules/clustering.html, 2024a.

F. Pedregosa et al. KMeans — scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html, 2024b.

F. Pedregosa et al. silhouette_score — scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html, 2024c.

F. Pedregosa et al. TargetEncoder — scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.TargetEncoder.html, 2024d.